

Technical Summary: Single-Source Shortest Paths with Negative Real Weights in $O(mn^{8/9})$ Time

Hussain Umer Farooqui, Hussain Mustansir

April 8, 2025

Problem and Contribution

The paper addresses the problem of computing single-source shortest paths (SSSP) on directed graphs with real edge weights, including both positive and negative values. The classic Bellman-Ford algorithm for this problem runs in $O(mn)$ time, where m is the number of edges and n is the number of vertices. This paper introduces a randomized algorithm that improves upon the Bellman-Ford algorithm by leveraging Dijkstra Algorithm, achieving a time complexity of $O(mn^{8/9})$, marking the first asymptotic improvement in the problem.

The main contribution of the paper is the development of a randomized algorithm that solves the SSSP problem for real-weighted graphs with high probability in the improved time complexity.

Algorithmic Description

Inputs

The input to the algorithm consists of:

- A directed graph $G = (V, E, w)$, where V is the set of vertices, E is the set of edges, and $w : E \rightarrow \mathbb{R}$ is a weight function assigning real weights (positive and negative) to the edges.
- A source vertex $s \in V$.

Outputs

The algorithm outputs:

- The shortest-path distances from the source vertex s to all other vertices in the graph, or
- It detects negative-weight cycles in the graph if there is a negative-weight cycle in the graph.

High-Level Logic

The algorithm follows a sequence of structured steps to efficiently compute the shortest paths while handling negative-weight edges. The core idea is to eliminate the impact of negative-weight edges gradually and reweight the graph to avoid them entirely. This is done in a series of well-defined steps:

1. **Initial Graph Analysis and Setup:** The input graph is first analyzed for negative-weight edges. These edges are key obstacles to the efficient computation of shortest paths. The graph is restructured and prepared for the following steps to progressively reduce their influence.
2. **Hop-Limited Shortest Paths (HLS):** The algorithm first uses a hop-limited shortest path (HLS) technique to identify paths that are limited by a number of negative edges, called "hops." This technique computes shortest paths while allowing for a limited number of negative edges in the path, helping to separate the influence of negative-weight edges from the rest of the graph. This helps to ensure that most of the graph can be handled without revisiting edges that would normally be influenced by negative weights.
3. **Price Function Reweighting:** Next, a *price function* $\phi : V \rightarrow \mathbb{R}$ is computed. The price function is a reweighting technique that adjusts the edge weights of the graph in such a way that all negative-weight edges are effectively eliminated. The price function achieves this by shifting the weights of edges, ensuring that no new negative-weight edges are created, and all previously negative-weight edges are transformed into non-negative weights. This reweighting allows for the use of efficient algorithms like Dijkstra's on the transformed graph.
4. **Randomized Elimination of Negative Edges:** The algorithm then applies a randomized $\Theta(k^{1/3})$ -elimination technique, where k is the number of negative-weight edges. This randomized approach efficiently eliminates a significant portion of the negative edges, reducing the graph's complexity. The main idea is that by progressively removing negative edges, the graph's structure becomes more conducive to applying standard shortest path algorithms (like Dijkstra's).
5. **Final Application of Dijkstra's Algorithm:** With the graph reweighted and negative edges reduced, the standard Dijkstra's algorithm is applied to find the shortest paths from the source to all other vertices. This step is performed on the transformed graph, which now only contains non-negative edges, making it suitable for Dijkstra's fast computation.

By using this multi-step process, the algorithm achieves a significant improvement in time complexity. The overall running time is reduced to $O(mn^{8/9})$.

Comparison with Existing Approaches

Bellman-Ford Algorithm

The Bellman-Ford algorithm is a well-known method for solving SSSP with real edge weights, running in $O(mn)$ time. This algorithm works by relaxing all edges iteratively.

The new approach significantly improves upon this by reducing the time complexity to $O(mn^{8/9})$, using randomized techniques and graph reweighting.

Dijkstra's Algorithm

For graphs with non-negative edge weights, Dijkstra's algorithm achieves an optimal time complexity of $O(m + n \log n)$. The new algorithm, while designed for real-weighted graphs, handles negative weights efficiently while maintaining a reduced time complexity.

Novelty

This approach is novel because it reduces the impact of negative-weight edges via a probabilistic reweighting strategy and hop-limited paths, significantly improving on the time complexity compared to the classical Bellman-Ford algorithm.

Data Structures and Techniques

Hop-Limited Shortest Paths

The algorithm utilizes hop-limited shortest paths to efficiently handle graphs with negative-weight edges. By limiting the number of negative edges a path can contain, the algorithm reduces the number of possible negative edges that need to be considered in the graph.

Price Functions

The price function ϕ is used to reweight the graph to eliminate negative edges. This function is computed based on shortest-path distances, ensuring that the reweighted graph does not contain negative-weight edges, thus making it amenable to Dijkstra's algorithm.

Random Sampling

The algorithm uses random sampling techniques to identify sets of negative edges. This probabilistic approach helps reduce the time complexity of the algorithm by focusing on the most relevant edges and vertices in the graph.

Implementation Outlook

Challenges

The main challenges in implementing this algorithm are:

- **Large Input Size:** For graphs with a large number of negative-weight edges, ensuring that the algorithm runs within the expected time complexity could be challenging, especially when the number of iterations increases.
- **Randomization:** Ensuring that the randomization is implemented correctly and consistently will be crucial for achieving the desired time complexity.